

Analyze the entire existing codebase of the built project thoroughly and perform a complete production-level code audit.

Your task is to deeply inspect all files, folders, configurations, dependencies, architecture, and implementations to identify:

- Errors and bugs
- Redundant code
- Duplicate files or duplicated logic
- Unused components, functions, variables, APIs, assets, or dependencies
- Garbage files, temporary files, unnecessary configurations, or leftover AI-generated content
- Dead code and unreachable logic
- Incorrect folder structure or bad project organization
- Performance bottlenecks
- Memory or resource inefficiencies
- Security vulnerabilities
- Inconsistent coding patterns
- Improper naming conventions
- Unoptimized imports and dependencies
- Broken routes, APIs, database queries, or integrations
- Incomplete implementations or placeholder code
- Debug/test code that should be removed in production
- Duplicate database schemas, migrations, or queries
- UI inconsistencies and unused frontend assets
- Serverless architecture issues
- Scalability or maintainability problems

Then:

- Clearly categorize all findings by severity:
 - Critical
 - Major
 - Minor
 - Optimization suggestions
- Provide:

- Exact file paths
- Explanation of each issue
- Why it is problematic
- Recommended fix
- Optimized implementation approach
- Detect whether any AI-generated code introduced:
 - Duplicate logic
 - Conflicting implementations
 - Unnecessary abstractions
 - Repeated dependencies
 - Overengineering
- Verify that the project follows:
 - Production-ready standards
 - Clean architecture principles
 - Scalable serverless practices
 - Proper Supabase integration
 - Proper Backblaze integration
 - Secure authentication and API handling
- Refactor and optimize wherever necessary while preserving existing functionality.
- Finally, generate:
 1. Complete audit report
 2. Cleanup plan
 3. Refactoring roadmap
 4. File/folder optimization suggestions
 5. Dependency cleanup recommendations
 6. Production-readiness score
 7. Final optimized architecture summary

Do not skip any file. Analyze the entire codebase deeply, including hidden configurations, build files, environment setup, deployment settings, and package dependencies.